DOCUMENTO DE LÓGICA DE NEGOCIO • VERSIÓN 1.0

FacturaHábiles CR

Cálculo automatizado de vencimientos con días hábiles, feriados y estados de facturas.

ESCENARIO	TECNOLOGÍAS	ALCANCE
01 · Central de facturación	JavaScript puro, n8n, JSON Server, Nager.Date	Facturas de prueba para Costa Rica

1 Problema que resuelve

Una empresa que vende a crédito necesita saber cuándo vence cada factura. Sumar 15 días naturales a la emisión puede producir fechas equivocadas, porque los sábados, domingos y feriados nacionales no deben contarse cuando el acuerdo es por días hábiles. Al revisar manualmente muchas facturas también se pueden omitir cobros atrasados o modificar registros que ya fueron pagados.

La automatización consulta las facturas almacenadas en JSON Server y los feriados nacionales publicados por Nager.Date. Después calcula la fecha de vencimiento, determina si corresponde el estado «pendiente» o «vencida» y persiste exclusivamente los cambios necesarios. Los datos de clientes son ficticios y solo sirven para demostrar el flujo.

2 Recorrido de los datos

ETAPA	OPERACIÓN EN n8n / RESPONSABILIDAD
01 INICIAR	Manual Trigger para demostración o Schedule Trigger diario a las 07:00, zona Costa Rica.
02 LEER	HTTP GET /facturas desde JSON Server; obtiene estado, fecha de emisión y plazo.
03 PLANIFICAR	Code: valida origen y detecta años necesarios, incluido el año siguiente si el plazo cruza diciembre.
04 DECIDIR	IF: si hay calendarios por consultar, continúa con la API; en otro caso emite resultado controlado.
05 ENRIQUECER	HTTP GET /PublicHolidays/{año}/CR desde Nager.Date; se comprueban los años recibidos.
06 CALCULAR	Code: CalendarioLaboral + Factura evalúan fechas, estados, errores e idempotencia.
07 GUARDAR	IF «guardar» → PATCH /facturas/{id}; sin cambios y alertas se reportan sin escribir.
08 COMPROBAR	El PATCH confirma el registro actualizado; JSON Server permite verificarlo mediante GET.

3 Reglas implementadas

Las reglas se aplican dentro de los métodos de las clases y, mediante generación automática, en los nodos Code del workflow. No se mantienen versiones manuales independientes.

#	REGLA	COMPORTAMIENTO VERIFICABLE
R1	Estados protegidos	Facturas pagadas o anuladas no se actualizan.
R2	Plazo hábil	La emisión no cuenta; se agregan entre 1 y 60 días hábiles posteriores.
R3	Calendario CR	No se cuentan sábados, domingos ni feriados de tipo Public y alcance nacional.
R4	Vencimiento	Solo se declara «vencida» cuando la fecha local actual es posterior al vencimiento; el mismo día continúa pendiente.
R5	Validación	El ID, el estado, el plazo y la fecha ISO deben ser válidos; una factura defectuosa no recibe PATCH.
R6	Datos completos	Se solicitan todos los años que puedan intervenir en el cómputo; un año faltante bloquea las escrituras.
R7	Idempotencia	Cuando fechaVencimiento y estado ya coinciden, el IF evita el PATCH.
R8	Persistencia	Solo se envían estado y fechaVencimiento; el resto de campos no se sustituye.

4 Manejo de situaciones inesperadas

Error de registro: la factura F-006 de prueba no contiene fechaEmision. El constructor de Factura provoca ERROR_VALIDACION, el registro se omite y las demás facturas válidas siguen su curso.

Error de API externa: si Nager.Date falla, devuelve datos inválidos o no se recibe un año requerido, el Code emite ERROR_API. El IF «guardar» toma la ruta negativa; no se ejecuta ningún PATCH. Es un diseño de fallo cerrado para evitar fechas incorrectas.

Error de origen: si JSON Server está apagado, el GET se maneja como ERROR_JSON_SERVER. La ejecución queda sin actualizaciones; si el servidor responde sin facturas, se informa SIN_FACTURAS. Un fallo aislado de PATCH también se informa como ERROR_ESCRITURA.

IMPLEMENTACIÓN, VALIDACIÓN Y CONCLUSIONES

5 POO y conexión con los nodos Code

La clase CalendarioLaboral recibe la colección de feriados y conoce dos operaciones: esHabil(fecha), que comprueba día de semana y feriados nacionales, y sumarDiasHabiles(fechaEmision, diasHabiles), que avanza fechas sin contar la emisión ni días no hábiles.

La clase Factura encapsula los datos y las reglas particulares de una factura. estaCerrada() identifica registros pagados o anulados. evaluar(calendario, hoyISO) obtiene la fecha de vencimiento, decide el estado y entrega la bandera guardar para que el IF determine si debe hacer PATCH. La función procesarFacturas coordina ambas clases y aísla errores de registros individuales.

El archivo src/negocio.js es la fuente única de la lógica. scripts/generar-workflow.js incrusta ese mismo código, de manera literal, dentro de los nodos Code y guarda su SHA-256. El comando npm run workflow:check verifica la identidad del módulo y las conexiones. main.js instancia las mismas clases sin arrancar n8n.

6 Decisiones técnicas

DECISIÓN	JUSTIFICACIÓN
Fechas ISO y operaciones UTC	Las fechas de calendario no se desplazan con el huso horario del servidor. La fecha de hoy se obtiene expresamente en America/Costa_Rica.
Consulta dinámica de años	Un plazo emitido en diciembre puede vencer en enero; los feriados de ambos años deben estar disponibles.
JSON Server para prácticas	Permite GET y PATCH visibles sin crear una base de datos de producción. El workflow opera sobre db.json mediante el servidor HTTP.
Nager.Date sin autenticación	Proporciona la lista de feriados públicos mediante HTTP. Se usa la clasificación Public de ámbito nacional.
Escrituras selectivas	Solo se actualiza cuando hay cambios; se evita trabajo duplicado y se facilita repetir la presentación.

7 Evidencia y límites de la entrega

Se ejecutaron 17 pruebas automatizadas: validaciones de fechas, feriados, cambio de año, estados, idempotencia y fallos de origen. Dos de ellas levantan un servidor HTTP local de prueba y verifican GET, evaluación POO y PATCH con persistencia simulada, además de un 503 de feriados con cero escrituras. Son pruebas locales, no acreditan una ejecución E2E dentro de n8n.

La comprobación final debe realizarse en el equipo de exposición: iniciar JSON Server, importar workflow/FacturaHabiles_CR_n8n.json, ejecutar n8n contra la API real y mostrar las respuestas y el db.json actualizado. La lista de Nager.Date debe contrastarse con disposiciones oficiales si se aplicara a obligaciones legales reales.

Conclusión: lo más difícil fue coordinar las reglas de negocio con servicios externos sin duplicar el código POO. La generación del workflow desde una única fuente y el bloqueo de escrituras ante datos incompletos resuelven ese riesgo.